

Callback → setTimeout → Promise → async/await → Fetch → JSON → localStorage.

1. Callbacks

Step 1 – Create a function

```
function sayHello() {  
  console.log("Hello");  
}
```

Step 2 – Create another function

```
function execute(fn) {  
  fn();  
}
```

Step 3 – Pass the function

```
execute(sayHello);
```

Expected Output

Hello

Explain

Here:

`execute(sayHello);`

We are **not calling** `sayHello`.

We are passing the function itself.

The receiving function decides when to execute it. This is the basic idea behind callbacks.

Example:

```
function add() {  
  console.log("Adding");  
}  
  
function subtract() {  
  console.log("Subtracting");  
}  
  
function execute(operation) {  
  operation();  
}  
  
execute(add);  
execute(subtract);
```

Expected:

Adding

Subtracting

2. Callback Functions

A callback is a function passed as an argument to another function and executed by that function.

Hands-on Example

```
function doTask(callback) {  
    console.log("Task started");  
    callback();  
}  
  
function finished() {  
    console.log("Task finished");  
}  
  
doTask(finished);
```

Output

Task started

Task finished

```
doTask(finished)  
  |  
  ↓  
Task started  
  |  
  ↓  
callback()  
  |  
  ↓  
finished()  
  |  
  ↓  
Task finished
```

Task

Change the program to:

Task started

Downloading...

Download completed

Task finished

3. Synchronous vs Asynchronous

"What happens if a task takes 5 seconds?"

Example:

```
Start Download
  ↓
  WAIT
  ↓
Download Complete
  ↓
Display Image
```

This is **synchronous-style waiting**.

Create:

```
console.log("1");

setTimeout(function() {
  console.log("2");
}, 3000);

console.log("3");
```

predict the output.

1

3

2

Important Question

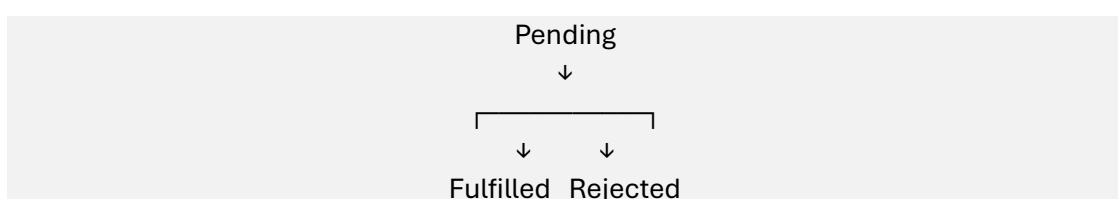
"Why didn't 2 come before 3?"

Because the setTimeout callback runs later.

4. Promise

A Promise represents the future result of an asynchronous operation.

Promise states:



Success	Failure
---------	---------

Step 1 – Create Promise

```
let promise = new Promise(function(resolve, reject) {  
  
  resolve("Data received");  
  
});
```

Step 2 – Get result

```
promise.then(function(result) {  
  
  console.log(result);  
  
});
```

Output

Data received

Step 3 – Promise with Success/Failure

```
function downloadImage() {  
  
  return new Promise(function(resolve, reject) {  
  
    console.log("Downloading JPG...");  
  
    let success = true;  
  
    if (success) {  
      resolve("photo.jpg");  
    }  
    else {  
      reject("Download failed");  
    }  
  
  });  
  
}
```

Then:

```
downloadImage()  
  .then(function(image) {  
  
    console.log("Display:", image);  
  
  });
```

```
})  
.catch(function(error) {  
  
  console.log(error);  
  
});
```

Change

```
let success = true;
```

to:

```
let success = false;
```

Observe the difference.

Explain

```
resolve()  
↓  
.then()  
  
reject()  
↓  
.catch()
```

5. Async and Await

.then() works, but long Promise chains can become difficult to read.

Step 1 – Create Promise

```
function getData() {  
  
  return new Promise(function(resolve) {  
  
    setTimeout(function() {  
  
      resolve("Data received");  
  
    }, 3000);  
  
  });  
  
}
```

Step 2 – Create async function

```
async function showData() {  
  
  let result = await getData();  
  
}
```

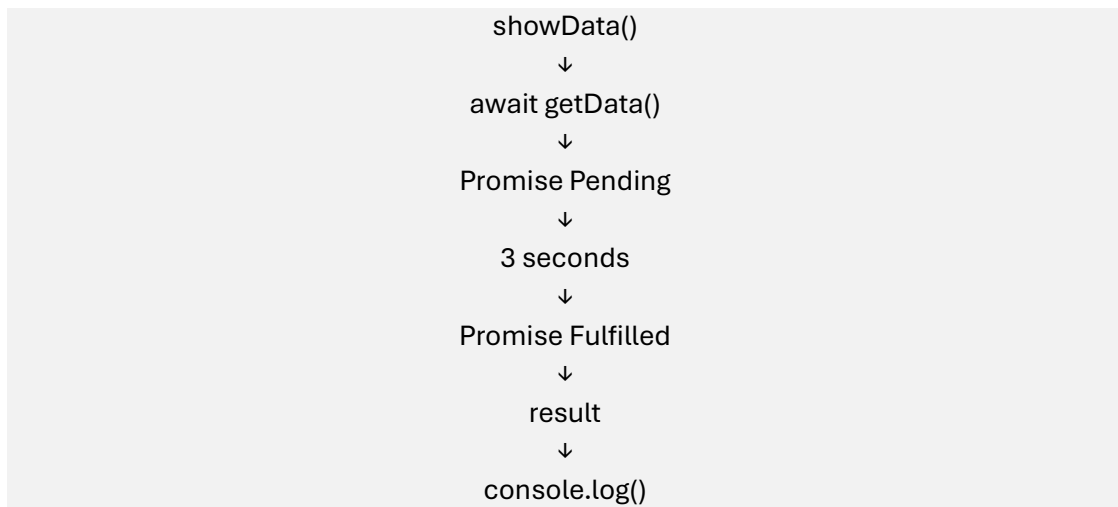
```
    console.log(result);  
  }  
  showData();
```

Ask students

"What does await do?"

answer:

It waits for the Promise to settle and gives us its result.



6. Async/Await Login

Login Promise

```
function login(username, password) {  
  return new Promise(function(resolve, reject) {  
    setTimeout(function() {  
      if (username === "admin" &&  
          password === "admin") {  
        resolve("Login Success");  
      }  
      else {  
        reject("Invalid Credentials");  
      }  
    }, 2000);
```

```
});  
  
}
```

Use async/await

```
async function checkLogin() {  
  
  try {  
  
    let result = await login("admin", "admin");  
  
    console.log(result);  
  
  }  
  catch(error) {  
  
    console.log("Invalid Credentials");  
  
  }  
  
}  
  
checkLogin();
```

Experiment

Change:

login("admin", "admin");

to:

login("admin", "123");

Expected:

Invalid Credentials

7. Fetch API + JSON

"Until now we created our own Promises. Now let's get data from a real server:"

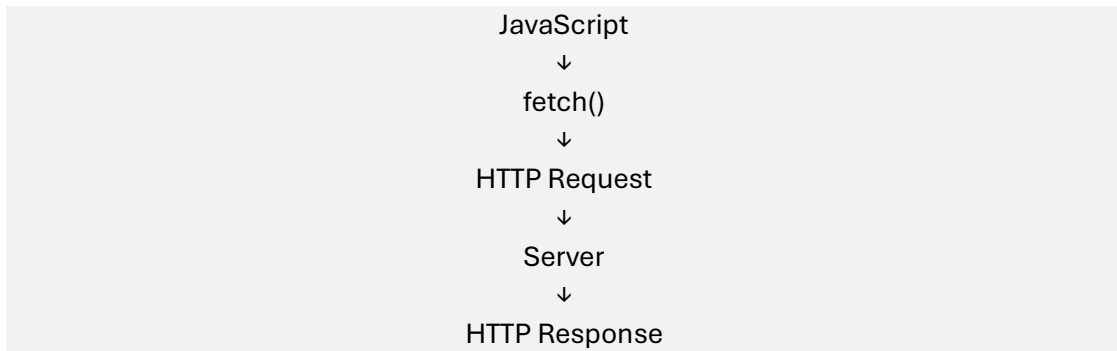
fetch() sends an HTTP request and returns a Promise.

Step 1 – Fetch data

```
async function getPost() {  
  
  let response =  
    await fetch("https://jsonplaceholder.typicode.com/posts/1");  
  
  console.log(response);  
  
}
```

```
}  
  
getPost();
```

Explain:



Step 2 – Convert Response to JSON

```
let data = await response.json();  
Complete code:  
async function getPost() {  
  
    let response =  
        await fetch("https://jsonplaceholder.typicode.com/posts/1");  
  
    let data = await response.json();  
  
    console.log(data);  
  
}  
  
getPost();
```

Step 3 – Display Individual Values

```
async function getPost() {  
  
    let response =  
        await fetch("https://jsonplaceholder.typicode.com/posts/1");  
  
    let data = await response.json();  
  
    console.log("ID:", data.id);  
    console.log("Title:", data.title);  
    console.log("Body:", data.body);  
  
}
```

```
getPost();
```

Expected Concept

The returned JSON contains fields such as:

id

title

body

8. Local Storage

browser storage covers:

- localStorage
- sessionStorage

Step 1 – Store Data

```
localStorage.setItem("username", "Ravi");
```

Step 2 – Retrieve Data

```
let username =  
  localStorage.getItem("username");  
  
console.log(username);
```

Output:

Ravi

Step 3 – Remove Data

```
localStorage.removeItem("username");
```

Step 4 – Clear Everything

```
localStorage.clear();
```

9. Store an Object Using JSON

This is especially important for the e-commerce project.

Create:

```
let user = {  
  name: "Ravi",  
  age: 25  
};
```

Store it:

```
localStorage.setItem(  
  "user",  
  JSON.stringify(user)  
);
```

Retrieve:

```
let data =  
  localStorage.getItem("user");
```

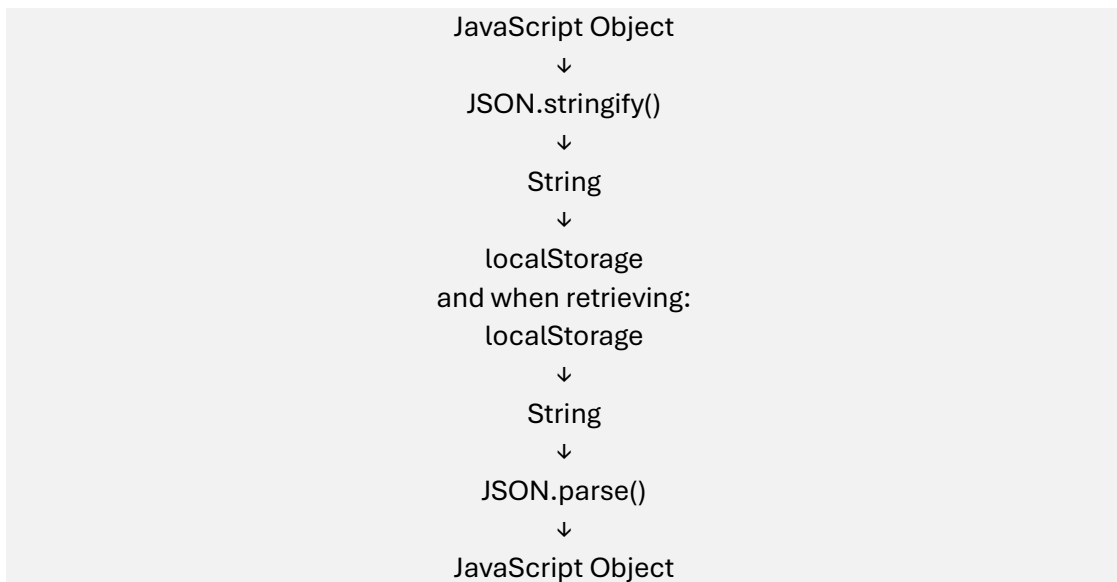
Convert back:

```
let userObject =  
  JSON.parse(data);
```

Display:

```
console.log(userObject.name);
```

Explain



10. Final 1-Minute Challenge

Create a program that fetches a post from the server and stores its title in Local Storage.

Expected Solution

```
async function getPost() {  
  
  let response =  
    await fetch("https://jsonplaceholder.typicode.com/posts/1");  
  
  let data = await response.json();  
  
  console.log("Title:", data.title);  
  
  localStorage.setItem("postTitle", data.title);  
}
```

```
}  
  
getPost();
```

Then retrieve it:

```
let title =  
  localStorage.getItem("postTitle");  
  
console.log("Stored Title:", title);
```

Key Syntax Students Must Remember

```
// Callback  
execute(functionName);  
  
// Promise  
new Promise(resolve, reject);  
  
// Promise result  
promise.then().catch();  
  
// Async/Await  
async function test() {  
  let result = await promise;  
}  
  
// Fetch  
let response = await fetch(url);  
  
// JSON  
let data = await response.json();  
  
// Local Storage  
localStorage.setItem(key, value);  
localStorage.getItem(key);  
localStorage.removeItem(key);  
localStorage.clear();
```